Wardrobe+

PROJECT PROPOSAL

SE380 Mobile Application Development

CORE PRODUCT IDEA

A Flutter/Dart mobile assistant that digitizes real wardrobes, recommends context-aware outfits, and reduces daily decision effort through user-verified AI classification and Smart Quick Swap.

DOCUMENT STATUS

Proposal - planned capabilities are not claimed as implemented

18 October 2026

1. Executive Summary

Wardrobe+ is a mobile application that helps users digitize their real wardrobes and make faster daily clothing decisions. Users add garments individually by photographing them, preferably against a plain white background. The application checks whether each image is sufficiently clear and well lit, then uses AI to suggest the garment's category, subcategory, color, suitable season, and style. The user remains in control: suggested classifications must be confirmed and may be corrected, while garment size is entered manually and optionally using fields appropriate to the garment category.

Using only items owned by the user, Wardrobe+ will generate several suitable daily outfits based on weather, occasion or destination, wearing history, and-outside the MVP-optional calendar information. Its distinguishing interaction, Smart Quick Swap, lets users tap one item in a proposed outfit and immediately choose from compatible replacements while all other items remain fixed. When the user selects "I Wore This," the application records the final edited outfit rather than the original suggestion.

The MVP focuses on the complete wardrobe-to-outfit loop. Advanced analytics and calendar integration are intentionally deferred to control scope.

2. Problem Statement

Choosing an outfit can require users to remember what they own, assess weather and context, coordinate multiple garments, and avoid repeatedly wearing the same items. Existing wardrobe records may organize clothing but still leave most of the decision-making to the user. Recommendation systems can also become inconvenient when changing one item requires rebuilding an entire outfit.

Wardrobe+ addresses these problems by:

1. making a user's physical wardrobe searchable in digital form;
2. reducing manual data entry through AI-assisted, user-verified classification;
3. recommending several context-appropriate outfits from owned garments;
4. reducing decision effort through item-level Smart Quick Swap;
5. recording what was actually worn for later frequency insights.

3. Project Objective

The objective is to design and implement a usable Flutter/Dart mobile application that demonstrates an end-to-end intelligent wardrobe workflow:

- capture and validate clothing images;
- construct an accurate, user-approved digital wardrobe;
- generate explainable and context-sensitive outfit suggestions;
- enable fast changes without forcing the user to browse the full wardrobe;
- persist the final outfit worn and favorite combinations;
- create reliable data for later weekly and monthly wardrobe analytics.

The project will evaluate whether this workflow can reduce outfit-selection effort while preserving user control over personal wardrobe data and AI-generated metadata.

4. Target Users

The primary target users are:

- students and young professionals who make frequent context-dependent outfit decisions;
- users who want to organize garments they already own;
- people seeking faster daily recommendations based on weather and occasion;
- users interested in understanding which garments and outfits they wear often or rarely.

The MVP assumes a single wardrobe owner per authenticated account. Social sharing, marketplaces, and multi-user wardrobes are outside the current scope.

5. Requirements

5.1 Functional Requirements - MVP

ID	Requirement
FR-01	The system shall allow a user to register, sign in, and access only their own wardrobe data.
FR-02	The user shall be able to photograph or upload each real garment individually, preferably against a plain white background.
FR-03	The system shall perform basic clarity and lighting quality checks and request a better image when minimum quality is not met.
FR-04	AI shall suggest category, subcategory, dominant color, suitable season, and style metadata.
FR-05	The user shall confirm or correct AI-suggested metadata before it is treated as authoritative wardrobe data.
FR-06	The user shall be able to enter an optional size manually; the available size format shall depend on the garment category where appropriate.
FR-07	The user shall be able to view and manage garments in a digital wardrobe.
FR-08	The system shall accept weather data and an occasion or destination context for outfit generation.
FR-09	The system shall produce several suitable daily outfit alternatives using the user's own garments.
FR-10	The user shall be able to tap one garment in a suggested outfit and see compatible alternatives for that position.
FR-11	Applying a Smart Quick Swap shall keep all non-selected outfit items fixed and update only the selected position.
FR-12	Selecting "I Wore This" shall record the latest edited version of the outfit with its date and component garments.
FR-13	The user shall be able to mark outfits as favorites and organize them under categories such as Casual, Business, Formal, and Sport.
FR-14	Stored wear records shall support later calculation of item and outfit usage frequency.

5.2 Post-MVP Requirements

The following are planned extensions, not commitments for the MVP:

- weekly and monthly analytics dashboards;
- clear identification of frequently and rarely worn garments or outfits;
- optional, permission-based calendar integration;
- automatic extraction of relevant occasion context from calendar events;
- richer personalization based on longer-term behavior.

5.3 Non-Functional Requirements

- Usability: the primary daily recommendation and swap workflow should require few, clear interactions.
- Performance: wardrobe browsing should feel responsive; remote AI and weather requests should provide visible loading, success, and failure states.
- Reliability: the system should not save an outfit as worn until the user explicitly confirms it.
- Maintainability: presentation, domain logic, data access, and external-service integration should be separated.
- Accessibility: controls should use readable contrast, meaningful labels, and adequate touch targets.
- Portability: the confirmed client technology is Flutter/Dart; supported mobile platforms will be finalized after course constraints and testing capacity are reviewed.
- Graceful degradation: wardrobe management and manual metadata correction should remain usable when AI or weather services are unavailable, subject to local/cache design decisions.

6. Core User Flow

1. The user creates an account or signs in.
2. The user photographs or uploads one garment, preferably against a plain white background.
3. Wardrobe+ checks image clarity and lighting.
4. If quality is insufficient, the user receives actionable feedback and retakes or replaces the image.
5. AI suggests category, subcategory, color, season, and style.
6. The user confirms or corrects the suggestions and may enter a category-appropriate size.
7. The garment is saved to the user's digital wardrobe.
8. For daily planning, the user provides an occasion or destination; weather is supplied automatically by a proposed weather integration or manually if needed.
9. The system proposes several outfits composed only of the user's garments.
10. The user may accept an outfit or tap one item to open Smart Quick Swap.
11. Compatible alternatives appear in a bottom sheet or equivalent compact UI. Selecting one replaces only that item.
12. The user may repeat swaps, save the outfit as a favorite, or select "I Wore This."
13. "I Wore This" stores the final edited outfit and updates the underlying wear history.
14. In a later phase, the user reviews weekly/monthly frequency insights.

7. Proposed Modules

Module	Responsibility	Status
Authentication	Registration, sign-in, session, and access control	MVP
Garment Capture	Camera/gallery intake and white-background guidance	MVP
Image Quality	Clarity and lighting checks with retry feedback	MVP
AI Classification	Metadata suggestions for category, subcategory, color, season, and style	MVP; service selection proposed
Classification Review	User confirmation and correction of AI output	MVP
Digital Wardrobe	Garment list, filters, details, edit, and removal	MVP
Size Management	Optional manual, category-aware size fields	MVP
Context Input	Weather and occasion/destination collection	MVP
Outfit Recommendation	Generation and ranking of several outfit options	MVP
Smart Quick Swap	Compatible alternatives while other garments remain fixed	MVP
Wear Journal	Persistence of the final edited outfit selected as worn	MVP
Favorites	Saved outfits grouped by use/style category	MVP
Analytics	Weekly/monthly item and outfit frequency insights	Post-MVP
Calendar Context	Optional, permission-based calendar integration	Post-MVP

8. Technology Proposal

8.1 Confirmed Technology

- Mobile client: Flutter
- Programming language: Dart

These choices are fixed because the project belongs to a Flutter/Dart mobile application course.

8.2 Proposed Architecture

A layered or clean architecture is proposed, separating presentation and state management, domain models and use cases, repository interfaces and data access, and integrations for image analysis, AI recommendation, weather, and optional calendar data.

The exact state-management library and package set remain to be evaluated against course requirements, team familiarity, and testing needs.

8.3 Candidate Backend and Services

These technologies are candidates rather than finalized commitments:

- Firebase Authentication for account management;
- Cloud Firestore for users, garments, outfits, favorites, and wear logs;
- Cloud Storage for Firebase for garment images;
- AI service for image metadata suggestions and/or outfit ranking;
- Weather API for current conditions and forecasts.

A small trusted server or serverless function is proposed if external AI/weather credentials must be protected or if recommendation logic should not run directly in the client. Final service selection will consider cost, free-tier limits, privacy, latency, implementation time, and course rules.

8.4 Preliminary Data Model

Proposed core entities include:

- User: profile and preference identifiers;
- Garment: owner, image reference, category, subcategory, color, season, style, optional size, AI suggestion provenance, and user-confirmation status;
- Outfit: component garment identifiers, context, creation source, and favorite category;
- WearLog: date/time, reference to the final outfit state, and its exact garment identifiers;
- RecommendationContext: weather summary, occasion/destination, and later optional calendar-derived context.

The wear log should preserve the exact final composition so later swaps or edits do not incorrectly change historical records.

9. Feasibility and Scope Control

The project is feasible as a course MVP if the team prioritizes a vertical slice before advanced intelligence:

1. reliable garment capture and storage;
2. user-verified metadata;
3. rule-based or hybrid outfit compatibility;
4. deterministic Smart Quick Swap constraints;
5. final-outfit wear logging;
6. basic favorites.

AI does not need to be trusted as the sole source of truth. User confirmation limits classification errors, while a rule-based baseline can support outfit generation and swapping if an advanced model is unavailable. Similarly, weather can initially be entered or mocked during development before a live provider is finalized.

To keep the scope realistic, the MVP excludes advanced analytics UI, calendar integration, social features, shopping, virtual try-on, automatic size detection, and guaranteed background removal. A plain white background is guidance for better image consistency; automated background removal is not a confirmed requirement.

10. Privacy and Security

Wardrobe images and wearing history are personal data. The proposed design will apply the following principles:

- authentication is required for private wardrobe access;
- authorization rules isolate each user's records and images;
- external API secrets must not be embedded in the Flutter application;
- only data required for a feature should be collected;
- calendar access, if introduced, must be optional and permission based;
- calendar processing should minimize stored event details;
- users should be informed when images or metadata are sent to an external AI provider;
- transport encryption and provider-supported encryption at rest should be used;
- logs must avoid tokens, private image URLs, and sensitive calendar content;
- deletion and retention behavior should be defined before production use.

A privacy notice, consent language, provider data-handling review, and account/data deletion flow are proposed before any public deployment.

11. Risks and Mitigations

Risk	Impact	Mitigation
Inconsistent photo quality	Poor classification and presentation	Capture guidance, clarity/lighting checks, and actionable retry feedback
AI metadata errors	Incorrect wardrobe records	Mandatory user review; editable fields; record confidence/provenance where useful
Weak outfit compatibility	Low user trust	Begin with explainable rules, test with curated wardrobes, then add AI ranking
Smart Swap breaks outfit context	Confusing results	Lock all unselected items and evaluate replacements against the complete fixed outfit
"I Wore This" logs the wrong version	Incorrect history	Persist an immutable snapshot of the final edited outfit at confirmation
External API downtime or latency	Blocked recommendations	Loading/error states, retries, caching where appropriate, and manual fallback
API cost or quota limits	Interrupted development/demo	Compare providers, enforce quotas, cache responses, and maintain a local/mock path
Exposed API credentials	Security and cost risk	Use a trusted backend/serverless proxy and secret management
Insufficient wardrobe data	Poor early recommendations	Seed test accounts and communicate minimum useful wardrobe coverage
Scope expansion	Missed course deadlines	Freeze MVP, time-box research, and defer analytics/calendar work
Personal-data leakage	Loss of trust	Per-user authorization rules, least-privilege access, secure URLs, and privacy review

12. Milestones

The following schedule is proposed and should be aligned with the official SE380 calendar.

Week	Milestone	Planned outcome
1	Research and proposal	Validate problem, inspect related solutions and technical options, finalize MVP boundaries, document risks, create proposal

Week	Milestone	Planned outcome
2	UX and technical design	User flows, wireframes, data model, architecture decision records, initial Flutter structure
3	Authentication and wardrobe foundation	Account flow, navigation, garment model, initial data persistence
4	Capture and quality feedback	Camera/gallery intake, photo guidance, clarity/lighting validation
5	AI-assisted classification	Suggested metadata, review/correction screen, optional size entry
6	Digital wardrobe	Browse, filter, inspect, edit, and remove garments
7	Context and recommendation baseline	Weather/occasion input and multiple outfit candidates
8	Smart Quick Swap	Item selection, ranked alternatives, fixed-item preservation
9	Wear journal and favorites	Final edited outfit logging and categorized favorites
10	Integration and testing	End-to-end tests, error states, authorization checks, usability fixes
11	Evaluation and stabilization	Test execution, performance checks, documented findings, defect resolution
12	Final delivery	Demonstration build, source code, documentation, presentation, and final report

13. Deliverables

Planned deliverables are:

- Flutter/Dart mobile application source code;
- runnable MVP build or documented run procedure;
- version-controlled project repository;
- project proposal and requirements documentation;
- architecture and data-model documentation;
- UI wireframes or design artifacts;
- configured backend/service documentation once choices are finalized;
- automated tests where practical and a manual end-to-end test checklist;
- evaluation results and known limitations;
- final report, presentation, and demonstration scenario.

At the proposal stage, only planning documentation is being delivered; application features must not be represented as complete until implemented and verified.

14. Evaluation Criteria

Functional completion

- A user can complete the full flow from sign-in to adding a garment, confirming metadata, receiving outfit suggestions, swapping an item, and logging the final outfit.
- All confirmed MVP requirements have demonstrable acceptance tests.
- "I Wore This" records the exact final edited composition.
- Smart Quick Swap changes only the selected garment position.

AI and recommendation quality

- Classification suggestions are compared with a labeled test set; category and key-attribute agreement are reported rather than assumed.
- Image-quality feedback is tested with intentionally clear, blurred, dark, and overexposed examples.
- Outfit and swap results are reviewed against defined weather, occasion, season, and compatibility rules.
- Users can always correct AI metadata.

Usability

- Representative users can complete the core flow without facilitator intervention.
- Task completion rate, observed errors, time on task, and a short satisfaction score are recorded.
- The number of interactions required to replace a suggested garment is measured.

Reliability, privacy, and engineering quality

- Authentication and per-user access rules are tested.
- External-service failure and empty-state behavior are demonstrated.
- Critical domain logic has automated tests where practical.
- No production secrets are stored in the client or repository.
- Known limitations and deferred features are documented transparently.

Exact numerical thresholds and the evaluation sample size remain to be finalized after Week 1 research and available participant/time constraints are known.

15. Week 1 Research and Proposal Work

Week 1 is a planned research and definition phase, not a claim of completed implementation. Its activities are:

1. review comparable digital wardrobe and outfit-planning applications;
2. identify the specific decision-effort problem Wardrobe+ will address;
3. define primary users and core scenarios;
4. confirm Flutter/Dart constraints and review relevant packages;
5. compare candidate backend, storage, AI, and weather options;
6. investigate practical clarity/lighting checks and white-background capture guidance;
7. define a garment taxonomy, including category-dependent size formats;
8. specify Smart Quick Swap invariants and ranking factors;
9. create initial user flow, data model, risk register, and privacy checklist;
10. freeze MVP versus post-MVP scope;
11. define testable evaluation criteria and a demonstration scenario;
12. review and approve this proposal before implementation planning.

Week 1 outputs

- approved project proposal;
- prioritized requirements and MVP boundary;
- preliminary user flow and architecture;
- candidate technology comparison with unresolved decisions clearly marked;
- initial research notes and references;
- project risk and privacy checklist;
- milestone plan and acceptance-test outline.

16. Scope Summary

Confirmed

- Flutter/Dart mobile client;
- real-garment photo capture/upload;
- image clarity and lighting checks;
- AI-suggested metadata with user confirmation/correction;
- optional manual, category-aware garment size;
- digital wardrobe;
- weather and occasion/destination context;
- several daily outfit suggestions;
- Smart Quick Swap with non-selected items fixed;
- final edited outfit logging through "I Wore This";
- categorized favorite outfits.

Deferred

- advanced weekly/monthly analytics presentation;
- optional calendar integration.

Proposed / To Be Evaluated

- Firebase Authentication;
- Cloud Firestore;
- Cloud Storage for Firebase;
- AI provider and model;
- weather provider;
- state-management library;
- detailed recommendation algorithm;
- supported mobile platform set and final deployment configuration.